

Preparing the Project for Azure Deployment

1 Comment / By Dot Net Tutorials / April 13, 2026

Back to: [Microsoft Azure Tutorials](#)

Preparing the Project for Azure Deployment

In the previous chapters, we learned what deployment means, why cloud hosting is useful, how an ASP.NET Core Web API works internally, and how to build a complete Product Management API using .NET 8, EF Core, and SQL Server. At this stage, our application is working properly on the local machine. Swagger is opening, the database connection is working, and CRUD operations are running successfully.

However, a project that works on a local machine is not automatically ready for Azure deployment. Local development and production deployment are two very different situations. On a developer machine, many settings are kept simple for convenience. But in production, the application must be safer, cleaner, easier to monitor, and easier to manage. That is why, before deploying to Azure, we must properly prepare the project.

In this chapter, we will improve the existing **ProductManagementApi** project to make it more production-ready. We will add testing, improve logging, separate configuration by environment, and make a few important changes expected in a real, deployment-ready application.

What We Will Improve in This Chapter

In this chapter, we will make the following practical improvements:

- Add an xUnit Test Project
- Add simple unit tests for the service layer
- Add Serilog logging
- Update Program.cs
- Improve appsettings files
- Keep Swagger only for the Development environment
- Prepare the project for the Release build and local publish

Why Do We Need to Prepare the Project Before Azure Deployment?

When we build an ASP.NET Core Web API locally, many things are usually configured only for development convenience. For example, the application may use a local SQL Server instance, Swagger may always be enabled, detailed error information may be visible, and sensitive values may still exist in configuration files. This is acceptable during development but not suitable for production.

A production-ready project should be:

- Secure

- Configurable
- Easy to publish
- Easy to monitor
- Easy to maintain
- Safe for real users and real data

So, preparing the project for Azure deployment means improving the application so that it can run safely and correctly outside the developer's machine.

Real-Time Understanding

We created our Product Management API in Chapter 3. On the local machine, the application currently uses:

- Local SQL Server
- Local configuration values
- Development-style logging
- Swagger for easy testing
- No automated unit tests

This setup is good for learning and local development. But when we move the same project to Azure, the environment changes.

In Azure:

- The API may run on Azure App Service
- The database may move to Azure SQL Database
- Production secrets should not come from source code
- The application should run with the production configuration
- Automated tests may run as part of a CI/CD pipeline

That is why we must prepare the project before deployment.

Why Do We Need Environment-Based Configuration?

Development and production are not the same. So, their configuration should also not be the same.

During development, we may use:

- Local SQL Server
- More detailed logs
- Local settings stored in development files

In production, we may use:

- Azure SQL Database
- Safer logging settings
- Values supplied from Azure configuration
- Stronger security practices

That is why ASP.NET Core supports environment-based configuration using files such as:

- appsettings.json
- appsettings.Development.json
- appsettings.Production.json

The main idea is simple:

- appsettings.json contains common settings
- appsettings.Development.json contains local development settings
- appsettings.Production.json contains production-friendly settings

This approach keeps the application flexible and makes deployment much easier.

Why Are We Adding Serilog Logging?

In Chapter 3, the project used the default ASP.NET Core logging providers. That is fine for local development and learning. But before production deployment, we usually want stronger and cleaner logging.

Serilog is a structured logging library. It helps us capture useful information about what the application is doing. For example, it can record when the application starts, when a request fails, and when an exception occurs. These logs become very useful when something goes wrong in production.

In simple words, Serilog helps us answer questions like:

- Did the application start correctly?
- Which request failed?
- What exception occurred?
- When did the problem happen?

That is why Serilog is a better fit as we prepare the project for production deployment.

Install Required NuGet Packages

In the **ProductManagementApi Project**, open **Package Manager Console** and run:

- **Install-Package Serilog.AspNetCore**
- **Install-Package Serilog.Sinks.File**
- **Install-Package Serilog.Sinks.Console**

- **Install-Package Serilog.Settings.Configuration**
- **Install-Package Serilog.Sinks.Async**

Why We Need These Packages

- **Serilog.AspNetCore** → Integrates Serilog with ASP.NET Core
- **Serilog.Sinks.File** → Writes logs to log files
- **Serilog.Sinks.Console** → Writes logs to log files
- **Serilog.Settings.Configuration** → Reads Serilog settings from appsettings.json.
- **Serilog.Sinks.Async**: Wraps logging sinks with asynchronous functionality to improve performance by offloading log processing to a background thread.

Update appsettings.json

This is the common configuration file. It contains settings that apply across all environments unless an environment-specific file overrides them. In our case, this file is a good place for shared logging settings and AllowedHosts.

```
{
  "Serilog": {
    "MinimumLevel": {
      "Default": "Information",
      "Override": {
        "Microsoft": "Warning",
        "Microsoft.AspNetCore": "Warning",
        "System": "Warning"
      }
    },
    "WriteTo": [
      {
        // Wrap sinks with the Async sink to enable asynchronous logging.
        "Name": "Async",
        "Args": {
          "configure": [
            {
              // Asynchronously log to a file with rolling, retention, and file
              size limit settings.
              "Name": "File",
              "Args": {
                // File sink configuration
                "path": "D:\\home\\LogFiles\\Application\\MyAppLog-.txt",
                "rollingInterval": "Day",
                "retainedFileCountLimit": 30,
                "fileSizeLimitBytes": 10485760, // 10 MB per file (10 * 1024 *
                1024 bytes)
                "rollOnFileSizeLimit": true,

```

```

        "outputTemplate": "{Timestamp:yyyy-MM-dd HH:mm:ss} [{Level:u3}]
[Application] {Message:l}{NewLine}{Exception}"
    }
},
{
    // Console sink configuration
    "Name": "Console",
    "Args": {
        "outputTemplate": "{Timestamp:yyyy-MM-dd HH:mm:ss} [{Level:u3}]
[Application] {Message:l}{NewLine}{Exception}"
    }
}
]
}
},
"Properties": {
    // Global properties to add additional information to each log event.
    "Application": "ProductManagementApi"
}
},
"AllowedHosts": "*"
}

```

Update appsettings.Development.json

This file is used only while the application is running in the Development environment. It is the right place for the local SQL Server connection string and other settings that are useful only on the developer machine.

```

{
  "ConnectionStrings": {
    "DefaultConnection": "Server=LAPTOP-
6P5NK25R\\SQLSERVER2022DEV;Database=ProductManagementDB;Trusted_Connection=True;Tru
  },
  "Serilog": {
    "MinimumLevel": {
      "Default": "Debug",
      "Override": {
        "Microsoft": "Warning",
        "Microsoft.AspNetCore": "Warning"
      }
    }
  }
}

```

Update appsettings.Production.json

This file is meant for production-friendly settings. It should not store real production secrets directly inside the source code. Instead, production values should usually come from Azure configuration.

```
{
  "Serilog": {
    "MinimumLevel": {
      "Default": "Information",
      "Override": {
        "Microsoft": "Warning",
        "Microsoft.AspNetCore": "Warning"
      }
    }
  }
}
```

Important Note:

In production, the connection string should come from:

- Azure App Service Configuration
- Environment Variables
- Azure Key Vault

So, the purpose of these files is not just organization. Their real purpose is to keep the application safe and adaptable across environments.

Why Should Production Secrets Not Be Hardcoded?

A hardcoded secret means writing a sensitive value directly in source code or in a file that may later be committed to source control.

Examples include:

- Database passwords
- API keys
- SMTP credentials
- Token signing secrets

This is dangerous because once sensitive data enters the source code, it can be copied, shared, committed to Git, or exposed by mistake.

A better approach is:

- Keep the code ready to read the configuration
- Store development values separately

- Supply production values from Azure App Service configuration, environment variables, or Azure Key Vault

This makes the application more secure and easier to manage.

What Are Environment Variables?

Environment variables are values provided to the application from outside the source code. The application reads them while running. This is useful because the same code can run across multiple environments without changing the source files.

For example:

- Development can use one connection string
- Testing can use another
- Production can use a completely different one

That is why environment variables are very important in real-world deployment. They help separate **code** from **configuration**.

Why Should Swagger Be Kept Only for Development?

Swagger is extremely useful during development because it helps us test endpoints quickly and understand the API structure. But in production, we usually do not want every public environment to expose the API testing UI by default. That is why it is common practice to enable Swagger only in the Development environment.

So, in simple words:

- Development → Swagger enabled
- Production → Swagger is usually disabled

This is a small but important production readiness step.

Replace Program.cs

Now open Program.cs and replace the full code with the following:

```
using Microsoft.EntityFrameworkCore;
using ProductManagementApi.Data;
using ProductManagementApi.Repositories;
using ProductManagementApi.Services;
using Serilog;

namespace ProductManagementApi
{
    public class Program
```

```

{
    public static void Main(string[] args)
    {
        var builder = WebApplication.CreateBuilder(args);

        // Create and configure the Serilog logger by reading settings
        // from appsettings.json, appsettings.Development.json, or
        // appsettings.Production.json based on the current environment.
        // This allows us to manage log levels, file sinks, and other logging
behavior
        // from configuration instead of hardcoding everything here.
        Log.Logger = new LoggerConfiguration()
            .ReadFrom.Configuration(builder.Configuration)
            .Enrich.FromLogContext()
            .CreateLogger();

        // Tell ASP.NET Core to use Serilog as the main logging provider
        // instead of the default built-in logging providers.
        builder.Host.UseSerilog();

        // Add controller support and keep JSON property names exactly
        // the same as the C# property names in our DTOs and models.
        builder.Services.AddControllers()
            .AddJsonOptions(options =>
            {
                options.JsonSerializerOptions.PropertyNamingPolicy = null;
            });

        // Add Swagger services so API endpoints can be tested easily
        // during development.
        builder.Services.AddEndpointsApiExplorer();
        builder.Services.AddSwaggerGen();

        // Register ApplicationDbContext and configure EF Core to use SQL Serv
        builder.Services.AddDbContext<ApplicationDbContext>(options =>

options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection"));

        // Register repository and service classes in the dependency injection
container.
        builder.Services.AddScoped<IProductRepository, ProductRepository>();
        builder.Services.AddScoped<IProductService, ProductService>();

        var app = builder.Build();

        // Enable Swagger UI only in Development environment.
        // This is useful for local testing but usually avoided in production.

```

```

        if (app.Environment.IsDevelopment())
        {
            app.UseSwagger();
            app.UseSwaggerUI();
        }

        // Logs details about every HTTP request such as:
        // request path, method, status code, and execution time.
        // This helps a lot when troubleshooting production issues.
        app.UseSerilogRequestLogging();

        // Redirect all HTTP requests to HTTPS
        app.UseHttpsRedirection();

        // Adds authorization middleware to the request pipeline
        app.UseAuthorization();

        // Maps controller endpoints to incoming HTTP requests
        app.MapControllers();

        // Starts the application
        app.Run();
    }
}
}

```

Why Are We Adding xUnit Tests Before Deployment?

Before deploying an application, it is a good practice to have at least a few automated tests. We are not trying to test every single line of code in this chapter. The goal is to add a small set of basic tests to verify the most important business logic before publishing the application.

In our project, the **ProductService** is a good place to start because it contains business logic. For example, it handles fetching product data, converting entities to DTOs, and processing create-or-update operations. If that logic breaks later, a simple unit test can detect the problem early.

So, the purpose of adding xUnit here is very practical. It gives us confidence that the service layer is behaving correctly before we move to Azure.

Create the xUnit Test Project in Visual Studio

Inside the same solution:

- Right-click the solution
- Click **Add**
- Click **New Project**

- Choose **xUnit Test Project**
- Name it **ProductManagementApi.Tests**
- Select **.NET 8**
- Click **Create**

Now add a project reference:

- Right-click **ProductManagementApi.Tests**
- Click **Add**
- Click **Project Reference**
- Select **ProductManagementApi**
- Click **OK**

This makes the test project able to access Product Services and Repositories.

Install the Moq Package in the Test Project:

Open the Package Manager Console and please execute the following command in the Test Project.

- **Install-Package Moq**

Add ProductServiceTests.cs

Create this file inside the test project:

```
using Microsoft.Extensions.Logging;
using Moq;
using ProductManagementApi.Models;
using ProductManagementApi.Repositories;
using ProductManagementApi.Services;

namespace ProductManagementApi.Tests
{
    public class ProductServiceTests
    {
        [Fact]
        public async Task
            GetAllProductsAsync_Returns_Mapped_ProductResponseDtos()
        {
            // Arrange
            // Create sample product data that will act as the fake data
            // returned by the repository during this test.
            var products = new List<Product>
            {
                new Product
            {
```

```

        Id = 1,
        Name = "Keyboard",
        Description = "Mechanical keyboard",
        Price = 2500,
        StockQuantity = 10,
        Category = "Electronics",
        IsActive = true,
        CreatedOn = DateTime.UtcNow
    }
};

// Create a mock repository so we can control what data
// is returned without depending on a real database.
var repositoryMock = new Mock<IProductRepository>();

// Configure the mocked repository to return our sample product list
// when GetAllAsync is called by the service.
repositoryMock.Setup(x => x.GetAllAsync())
    .ReturnsAsync(products);

// Create a mock logger because ProductService expects an
ILogger<ProductService>
// in its constructor, but we do not need real logging behavior in
this test.
var loggerMock = new Mock<ILogger<ProductService>>();

// Create the ProductService instance by injecting the mocked
repository
// and mocked logger.
var service = new ProductService(repositoryMock.Object,
loggerMock.Object);

// Act
// Call the service method that we want to test.
var result = await service.GetAllProductsAsync();

// Assert
// Verify that exactly one product response DTO is returned.
Assert.Single(result);

// Verify that the returned product data is mapped correctly
Assert.Equal("Keyboard", result[0].Name);
Assert.Equal("Electronics", result[0].Category);
Assert.Equal(2500, result[0].Price);
    }
}
}

```

Add ProductServiceGetByIdTests.cs

Create this file inside the test project:

```
using Microsoft.Extensions.Logging;
using Moq;
using ProductManagementApi.Models;
using ProductManagementApi.Repositories;
using ProductManagementApi.Services;

namespace ProductManagementApi.Tests
{
    public class ProductServiceGetByIdTests
    {
        [Fact]
        public async Task
GetProductByIdAsync_Returns_Null_When_Product_Does_Not_Exist()
        {
            // Arrange
            // Create a mock repository so we can simulate repository behavior
            // without connecting to a real database.
            var repositoryMock = new Mock<IProductRepository>();

            // Configure the mocked repository to return null when the service
            // requests a product with Id 999. This simulates a "product not
found" case.
            repositoryMock.Setup(x => x.GetByIdAsync(999))
                .ReturnsAsync((Product?)null);

            // Create a mock logger because ProductService requires an
ILogger<ProductService>
            // in its constructor, but real logging is not needed for this unit
test.
            var loggerMock = new Mock<ILogger<ProductService>>();

            // Create the service instance by injecting the mocked repository
            // and mocked logger.
            var service = new ProductService(repositoryMock.Object,
loggerMock.Object);

            // Act
            // Call the service method with a product Id that does not exist.
            var result = await service.GetProductByIdAsync(999);

            // Assert
            // Verify that the service correctly returns null
            // when the requested product is not found.
        }
    }
}
```

```
        Assert.Null(result);  
    }  
}  
}
```

Why Are We Using Release Mode for Deployment?

When developing locally, we often work in **Debug** mode because it is useful for coding and debugging.

But when preparing for deployment, we should use **Release** mode. The simple difference is:

- **Debug** is for development and troubleshooting
- **Release** is for deployment

Release mode provides the build intended for hosting. So before publishing to Azure, we should always build the project in Release mode and verify that everything still works correctly.

What Is the Difference Between Build and Publish?

This is one area where many beginners get confused.

- **Build:** Build compiles the source code and checks whether the project can be compiled successfully.
- **Publish:** Publish does more than build. It prepares the deployable output of the application. This output contains the files that Azure or another hosting environment will actually use to run the application.

So, in simple words:

- **Build** checks whether the project compiles
- **Publish** prepares the project for deployment

That is why local publish is a very useful step before Azure deployment.

Why Should We Publish Locally Once Before Azure?

Publishing locally helps us verify that the application is really deployment-ready.

It helps confirm:

- The Release build works correctly
- Required files are generated properly
- Configuration is not broken
- Dependencies are available
- The project structure is ready for deployment

So, local publish acts like a small final check before moving to Azure.

Publish the Project Locally

Before going to Azure, it is a very good idea to test local publishing.

Using Visual Studio:

- Right-click the project
- Click **Publish**
- Choose **Folder**
- Select a local publish folder
- Use **Release** configuration
- Publish the project

What Actually Changes from Chapter 3 to Chapter 4?

This is a very important point for beginners.

- In Chapter 3, the application was successfully created and runs on the local machine. That was the goal of that chapter.
- In Chapter 4, the goal changes. We are not just trying to run the application locally now. We are preparing it for production deployment.

So, compared to Chapter 3, Chapter 4 introduces these improvements:

- Better configuration management
- Automated unit testing
- Structured logging with Serilog
- Safer handling of production settings
- Development-only Swagger
- Release build and local publish validation

This is the transition from **a local development project** to a **deployment-ready project**.

Summary

In this chapter, we prepared our existing Product Management API for Azure deployment. We understood that a project working on the local machine is not automatically ready for production. Before deployment, the application must be improved to make it safer, cleaner, easier to test, and easier to monitor.

We introduced xUnit testing to validate important service-layer logic before deployment. We added Serilog so that the application can produce better logs. We also improved environment-based configuration so that development and production settings remain separate.

Finally, we understood why Release build, local publish, environment variables, and secure secret handling are important in real-world deployment. With these improvements, our project is now much closer to being production-ready for Azure.



Dot Net Tutorials

About the Author: Pranaya Rout

Pranaya Rout has published more than 3,000 articles in his 11-year career. Pranaya Rout has very good experience with Microsoft Technologies, Including C#, VB, ASP.NET MVC, ASP.NET Web API, EF, EF Core, ADO.NET, LINQ, SQL Server, MYSQL, Oracle, ASP.NET Core, Cloud Computing, Microservices, Design Patterns and still learning new technologies.

Privacy Preferences

We and our partners share information on your use of this website to help improve your experience. For more information, or to opt out click the Do Not Sell My Information button below.

Consent

Do Not Sell My Information